```
class MyClass; end
obj1 = MyClass.new
obj2 = MyClass.new
```

The diagram shows some of the connections between the program entities. Now it's your turn to add more lines and boxes to the diagram and answer these questions:

- What's the class of Object?
- What's the superclass of Module?
- What's the class of Class?
- Imagine that you execute this code:

  ```
  obj3 = MyClass.new
  obj3.instance_variable_set("@x", 10)
  ```

  Can you add obj3 to the diagram?

You can use irb and the Ruby documentation to find out the answers.

Quiz Solution

Your enhanced version of the original diagram is in Figure 3, *Bill's diagram, enhanced by you*, on page 28.

As you can easily check in irb, the superclass of Module is Object. You don't even need irb to know what the class of Object is: because Object is a class, its class must be Class. This is true of all classes, meaning that the class of Class must be Class itself. Don't you love self-referential logic?

Finally, calling instance_variable_set blesses obj3 with its own instance variable @x. If you find this concept surprising, remember that in a dynamic language such as Ruby, every object has its own list of instance variables, independent of other objects—even other objects of the same class.

What Happens When You Call a Method?

Where you learn that a humble method call requires a lot of work on Ruby's part and you shed light on a twisted piece of code.

After some hours working on Bookworm, you and Bill already feel confident enough to fix some minor bugs here and there—but now, as your working day is drawing to a close, you find yourself stuck. Attempting to fix a long-standing bug, you've stumbled upon a tangle of classes, modules, and methods that you can't make heads or tails of.